Heart Disease Prediction System

🌐 Live Deployment Link: https://heartdiseaseprediction-omqtad3saltpkdqo4ayxtn.streamlit.app/

💻 GitHub Repository: https://github.com/Hamna-Mubarak/Heart_Disease_Prediction

Student Name	Hamna Mubarak
Roll No	FA24-BSE-046
Section	A
Course	Machine Learning Fundamentals (AIC354)
Instructor	Dr. Rao Muhammad Adeel Nawab
Assignment	Assignment 2: Developing an End-to-End Machine Learning System (Task 2)
Institute	COMSATS University Islamabad, Lahore Campus

Setup: Running This Notebook From Scratch

Run these commands in a terminal, inside this project's folder, before opening the notebook.

1. Create a virtual environment

```
python -m venv venv
```

2. Activate it

- Windows (Command Prompt): `venv\Scripts\activate.bat`
- Windows (PowerShell): `venv\Scripts\Activate.ps1`
- macOS / Linux: `source venv/bin/activate`

3. Install the required packages

```
pip install -r requirements.txt
```

4. Launch Jupyter Notebook, open this `.ipynb` file and run **Kernel -> Restart & Run All**.

```
jupyter notebook
```

The dataset file `processed_cleveland.csv` must be in the same folder as this notebook.

SU Meta Description

A Heart Disease Prediction System that uses a Support Vector Classifier trained on the UCI Cleveland Heart Disease dataset to predict whether a patient has heart disease from four clinical attributes.

Introduction

Definition A binary classification ML task that predicts whether a patient has heart disease (Yes) or does not have heart disease (No) from their clinical attributes (e.g., sex, chest pain type, number of major vessels, thalassemia test result).

Application It can be used as a decision-support / screening tool to help doctors flag high-risk patients for further tests, and for learning how clinical attributes relate to heart disease.

Dataset

Name	Heart Disease Dataset (Cleveland subset)
Source	UCI Machine Learning Repository — https://archive.ics.uci.edu/dataset/45/heart+disease
File used	`processed.cleveland.data` saved as `processed_cleveland.csv` (header row added, values unchanged)
Instances	303 patients (297 after removing rows with missing values)
Attributes	13 input attributes + 1 output attribute (`num` , 0 = no disease, 1–4 = disease)

Input and Output

As required by the assignment, **only 4 input attributes** are selected from the dataset.

Inputs (Features / Attributes)

Input Feature/Attribute	UCI Column	Possible Values
Sex	`sex`	Male, Female
ChestPainType	`cp`	Typical Angina, Atypical Angina, Non-Anginal Pain, Asymptomatic
MajorVessels (number of major vessels coloured by fluoroscopy)	`ca`	Zero, One, Two, Three
Thal (thallium stress test result)	`thal`	Normal, Fixed Defect, Reversible Defect

Output (Prediction)

Output Attribute	Possible Values
HeartDisease	Yes, No

Examples (Heart Disease Dataset)

Sex	ChestPainType	MajorVessels	Thal	HeartDisease
Male	Typical Angina	Zero	Fixed Defect	No
Male	Asymptomatic	Three	Normal	Yes
Male	Asymptomatic	Two	Reversible Defect	Yes
Male	Non-Anginal Pain	Zero	Normal	No
Female	Atypical Angina	Zero	Normal	No

Developer & System Information

Developer Name	Hamna Mubarak (FA24-BSE-046, Section A)
Based on	Titanic Passenger Survival Prediction System by Mr. Mohsin Afzal & Dr. Rao Muhammad Adeel Nawab
Program Name	heart_disease_project
IDE	Jupyter Notebook
Programming Language	Python 3.12
Libraries	NumPy 2.2.6, Pandas 2.3.3, Pickle (built-in), scikit-learn 1.7.2, PrettyTable 3.18.0, Matplotlib 3.10.9, Seaborn 0.13.2
Deployment	Streamlit Community Cloud
Date of Completion	10-Oct-2026

Table of Content

- Step 1: Import Libraries
- Step 2: Load Sample Data
- Step 3: Understand and Pre-process Sample Data
  - Step 3.1: Understand Sample Data
  - Step 3.2: Pre-process Sample Data
  - Step 3.3: Exploratory Data Analysis (EDA)
- Step 4: Feature Extraction (Select 4 Input Attributes)
- Step 5: Label Encoding the Sample Data (Input and Output is converted in Numeric Representation)
  - Step 5.1: Train the Label Encoder
  - Step 5.2: Label Encode the Output
  - Step 5.3: Label Encode the Input
- Step 6: Execute the Training Phase
  - Step 6.1: Splitting Sample Data into Training Data and Testing Data
  - Step 6.2: Splitting Input Vectors and Outputs / Labels of Training Data

    - Step 6.3: Train the Support Vector Classifier
    - Step 6.4: Save the Trained Model
- Step 7: Execute the Testing Phase
    - Step 7.1: Splitting Input Vectors and Outputs/Labels of Testing Data
    - Step 7.2: Load the Saved Model
    - Step 7.3: Evaluate the Machine Learning Model
        - Step 7.3.1: Make Predictions with the Trained Model on Testing Data
    - Step 7.4: Calculate the Accuracy Score
    - Step 7.5: Detailed Evaluation Metrics (Precision, Recall, F1, Confusion Matrix)
    - Step 7.6: 10-Fold Cross Validation (Robustness Check)
- Step 8: Execute the Application Phase
    - Step 8.1: Take Input from User
    - Step 8.2: Convert User Input into Feature Vector
    - Step 8.3: Label Encoding of Feature Vector
    - Step 8.4: Load the Saved Model
    - Step 8.5: Model Prediction
        - Step 8.5.1: Apply Model and Return Prediction to the User
- Step 9: Execute the Feedback Phase
    - Step 9.1: Collect Feedback from Users and Domain Experts
    - Step 9.2: Make a List of Potential Improvements
    - Step 9.3: Improve the Model Based on Feedback
- Step 10: Deploy the Model to the Cloud (Streamlit Community Cloud)

Code – Heart Disease Prediction

Step 1: Import Libraries

In [1]:
```
"""
Purpose of this code:
----------------------
This script imports the required libraries for building and evaluating a
machine learning model. It sets up tools for data manipulation, preprocessing,
splitting datasets, training a Support Vector Machine (SVM) model,
```

```
evaluating accuracy, visualising data, and displaying results in tabular form.
"""

# Import numerical computing library (used for arrays, math operations, etc.)
import numpy as np

# Import data manipulation library (used for DataFrames, reading/writing data, etc.)
import pandas as pd

# Import pickle (used for saving and loading Python objects/models)
import pickle

# Import train_test_split (used to divide dataset into training and testing sets)
# and cross_val_score (used for k-fold cross validation)
from sklearn.model_selection import train_test_split, cross_val_score

# Import LabelEncoder (used to convert categorical values into numeric codes)
from sklearn.preprocessing import LabelEncoder

# Import Support Vector Machine (SVM) model from scikit-learn
from sklearn import svm

# Import evaluation metrics
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Import PrettyTable (used to display results in a clean, table format)
from prettytable import PrettyTable

# Import plotting libraries (used for EDA charts and the confusion matrix heatmap)
import matplotlib.pyplot as plt
import seaborn as sns

# Colours used in all charts: blue = No heart disease, orange = Yes heart disease
COLOR_NO = "#2a78d6"
COLOR_YES = "#eb6834"
```

Step 2: Load Sample Data

In [2]:

```
"""
Purpose of this section:
-------------------------
This part of the code loads the UCI Cleveland Heart Disease dataset from a CSV
file into a pandas DataFrame. The original UCI file has no header row, so the
14 attribute names from the UCI documentation are used as column names.
Missing values in the UCI file are written as '?'.
"""

# Read the CSV file and load it into a pandas DataFrame
# 'processed_cleveland.csv' should be present in the working directory
raw_data = pd.read_csv("processed_cleveland.csv")
```

```python
# Print a header to indicate that sample data is being displayed
print("\n\nSample Data (first 10 rows):")
print("============================\n")

# Configure pandas to display all columns (so nothing gets hidden with '...')
pd.set_option("display.max_columns", None, "display.width", 200)

# Print the first 10 rows of the dataset
print(raw_data.head(10))

print("\nShape of Sample Data (rows, columns):", raw_data.shape)
```

```
Sample Data (first 10 rows):
============================

    age  sex   cp  trestbps   chol  fbs  restecg  thalach  exang  oldpeak  slope   ca thal  num
0  63.0  1.0  1.0     145.0  233.0  1.0      2.0    150.0    0.0      2.3    3.0  0.0  6.0    0
1  67.0  1.0  4.0     160.0  286.0  0.0      2.0    108.0    1.0      1.5    2.0  3.0  3.0    2
2  67.0  1.0  4.0     120.0  229.0  0.0      2.0    129.0    1.0      2.6    2.0  2.0  7.0    1
3  37.0  1.0  3.0     130.0  250.0  0.0      0.0    187.0    0.0      3.5    3.0  0.0  3.0    0
4  41.0  0.0  2.0     130.0  204.0  0.0      2.0    172.0    0.0      1.4    1.0  0.0  3.0    0
5  56.0  1.0  2.0     120.0  236.0  0.0      0.0    178.0    0.0      0.8    1.0  0.0  3.0    0
6  62.0  0.0  4.0     140.0  268.0  0.0      2.0    160.0    0.0      3.6    3.0  2.0  3.0    3
7  57.0  0.0  4.0     120.0  354.0  0.0      0.0    163.0    1.0      0.6    1.0  0.0  3.0    0
8  63.0  1.0  4.0     130.0  254.0  0.0      2.0    147.0    0.0      1.4    2.0  1.0  7.0    2
9  53.0  1.0  4.0     140.0  203.0  1.0      2.0    155.0    1.0      3.1    3.0  0.0  7.0    1

Shape of Sample Data (rows, columns): (303, 14)
```

Step 3: Understand and Pre-process Sample Data

Step 3.1: Understand Sample Data

Meaning of the 14 attributes (from the UCI documentation):

Column	Meaning	Column	Meaning
`age`	Age in years	`thalach`	Maximum heart rate achieved
`sex`	1 = male, 0 = female	`exang`	Exercise induced angina (1 = yes)
`cp`	Chest pain type (1 = typical angina, 2 = atypical angina, 3 = non-anginal pain, 4 = asymptomatic)	`oldpeak`	ST depression induced by exercise

Column	Meaning	Column	Meaning
`trestbps`	Resting blood pressure (mm Hg)	`slope`	Slope of the peak exercise ST segment
`chol`	Serum cholesterol (mg/dl)	`ca`	Number of major vessels (0–3) coloured by fluoroscopy
`fbs`	Fasting blood sugar > 120 mg/dl (1 = true)	`thal`	3 = normal, 6 = fixed defect, 7 = reversible defect
`restecg`	Resting ECG results (0, 1, 2)	`num`	**Output**: 0 = no disease, 1–4 = disease present

In [3]:

```
"""
Purpose of this section:
------------------------
This part of the code helps us understand the dataset by:
1. Printing the names of all attributes (columns) in the DataFrame.
2. Showing the total number of instances (rows) available in the dataset.
3. Showing the data type of each attribute.
4. Finding missing values (written as '?' in the UCI file).
5. Showing the distribution of the output attribute 'num'.
"""

# Display all column names (attributes) in the dataset
print("\n\nAttributes in Sample Data:")
print("==========================\n")
print(raw_data.columns)

# Total number of instances
print("\n\nNumber of Instances in Sample Data:", raw_data["age"].count())
print("========================================\n")

# Data types: 'ca' and 'thal' are read as text (object) because they contain '?'
print("\nData Types of Attributes:")
print("=========================\n")
print(raw_data.dtypes)

# Count the missing values ('?') in each attribute
print("\n\nMissing Values ('?') in each Attribute:")
print("=======================================\n")
print((raw_data == "?").sum())

# Distribution of the output attribute
print("\n\nDistribution of Output Attribute 'num' (0 = no disease, 1-4 = disease):")
print("=======================================================================\n")
print(raw_data["num"].value_counts().sort_index())
```

```
Attributes in Sample Data:
==========================

Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'num'], dtype='object')


Number of Instances in Sample Data: 303
=======================================


Data Types of Attributes:
=========================

age         float64
sex         float64
cp          float64
trestbps    float64
chol        float64
fbs         float64
restecg     float64
thalach     float64
exang       float64
oldpeak     float64
slope       float64
ca           object
thal         object
num           int64
dtype: object


Missing Values ('?') in each Attribute:
=======================================

age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          4
thal        2
num         0
dtype: int64


Distribution of Output Attribute 'num' (0 = no disease, 1-4 = disease):
```

```
======================================================================

num
0    164
1     55
2     36
3     35
4     13
Name: count, dtype: int64
```

In [4]:
```
"""
Purpose of this section:
------------------------
Summary statistics (count, mean, std, min, max, quartiles) of the numeric attributes.
"""

print("\n\nSummary Statistics of Sample Data:")
print("==================================\n")
print(raw_data.describe().round(2))
```

```
Summary Statistics of Sample Data:
==================================

          age     sex      cp  trestbps    chol     fbs  restecg  thalach   exang  oldpeak   slope     num
count  303.00  303.00  303.00    303.00  303.00  303.00   303.00   303.00  303.00   303.00  303.00  303.00
mean    54.44    0.68    3.16    131.69  246.69    0.15     0.99   149.61    0.33     1.04    1.60    0.94
std      9.04    0.47    0.96     17.60   51.78    0.36     0.99    22.88    0.47     1.16    0.62    1.23
min     29.00    0.00    1.00     94.00  126.00    0.00     0.00    71.00    0.00     0.00    1.00    0.00
25%     48.00    0.00    3.00    120.00  211.00    0.00     0.00   133.50    0.00     0.00    1.00    0.00
50%     56.00    1.00    3.00    130.00  241.00    0.00     1.00   153.00    0.00     0.80    2.00    0.00
75%     61.00    1.00    4.00    140.00  275.00    0.00     2.00   166.00    1.00     1.60    2.00    2.00
max     77.00    1.00    4.00    200.00  564.00    1.00     2.00   202.00    1.00     6.20    3.00    4.00
```

Step 3.2: Pre-process Sample Data

o Replace the missing values `'?'` with `NaN` and **remove the 6 rows** that contain missing values (4 in `ca` , 2 in `thal`)

o Convert the output attribute `num` (0–4) into a binary output **HeartDisease**: 0 → **No**, 1–4 → **Yes**

o Convert the numeric codes of the categorical attributes into readable text values (same style as the Titanic data, e.g. `sex` 1 → **Male**)

In [5]:
```
"""
Purpose of this section:
------------------------
This part of the code cleans the raw UCI data so that it has the same form as
the Titanic sample data (readable categorical values, no missing values).
"""
```

```python
# 1. Replace '?' with NaN, convert columns back to numbers and drop rows with missing values
clean_data = raw_data.replace("?", np.nan)
clean_data["ca"] = pd.to_numeric(clean_data["ca"])
clean_data["thal"] = pd.to_numeric(clean_data["thal"])

print("Rows before removing missing values:", len(clean_data))
clean_data = clean_data.dropna().reset_index(drop=True)
print("Rows after removing missing values :", len(clean_data))

# 2. Convert the output 'num' (0-4) into binary Yes / No
clean_data["HeartDisease"] = np.where(clean_data["num"] > 0, "Yes", "No")

# 3. Convert numeric codes into readable categorical values
clean_data["Sex"] = clean_data["sex"].map({1: "Male", 0: "Female"})
clean_data["ChestPainType"] = clean_data["cp"].map({
    1: "Typical Angina", 2: "Atypical Angina", 3: "Non-Anginal Pain", 4: "Asymptomatic"})
clean_data["MajorVessels"] = clean_data["ca"].map({0: "Zero", 1: "One", 2: "Two", 3: "Three"})
clean_data["Thal"] = clean_data["thal"].map({3: "Normal", 6: "Fixed Defect", 7: "Reversible Defect"})

print("\n\nPre-processed Data (first 10 rows):")
print("===================================\n")
print(clean_data[["sex", "Sex", "cp", "ChestPainType", "ca", "MajorVessels",
                  "thal", "Thal", "num", "HeartDisease"]].head(10))

print("\n\nOutput Distribution after Pre-processing:")
print("=========================================\n")
print(clean_data["HeartDisease"].value_counts())
```

```
Rows before removing missing values: 303
Rows after removing missing values : 297


Pre-processed Data (first 10 rows):
====================================
```

	sex	Sex	cp	ChestPainType	ca	MajorVessels	thal	Thal	num	HeartDisease
0	1.0	Male	1.0	Typical Angina	0.0	Zero	6.0	Fixed Defect	0	No
1	1.0	Male	4.0	Asymptomatic	3.0	Three	3.0	Normal	2	Yes
2	1.0	Male	4.0	Asymptomatic	2.0	Two	7.0	Reversible Defect	1	Yes
3	1.0	Male	3.0	Non-Anginal Pain	0.0	Zero	3.0	Normal	0	No
4	0.0	Female	2.0	Atypical Angina	0.0	Zero	3.0	Normal	0	No
5	1.0	Male	2.0	Atypical Angina	0.0	Zero	3.0	Normal	0	No
6	0.0	Female	4.0	Asymptomatic	2.0	Two	3.0	Normal	3	Yes
7	0.0	Female	4.0	Asymptomatic	0.0	Zero	3.0	Normal	0	No
8	1.0	Male	4.0	Asymptomatic	1.0	One	7.0	Reversible Defect	2	Yes
9	1.0	Male	4.0	Asymptomatic	0.0	Zero	7.0	Reversible Defect	1	Yes

```
Output Distribution after Pre-processing:
==========================================

HeartDisease
No     160
Yes    137
Name: count, dtype: int64
```

Step 3.3: Exploratory Data Analysis (EDA)

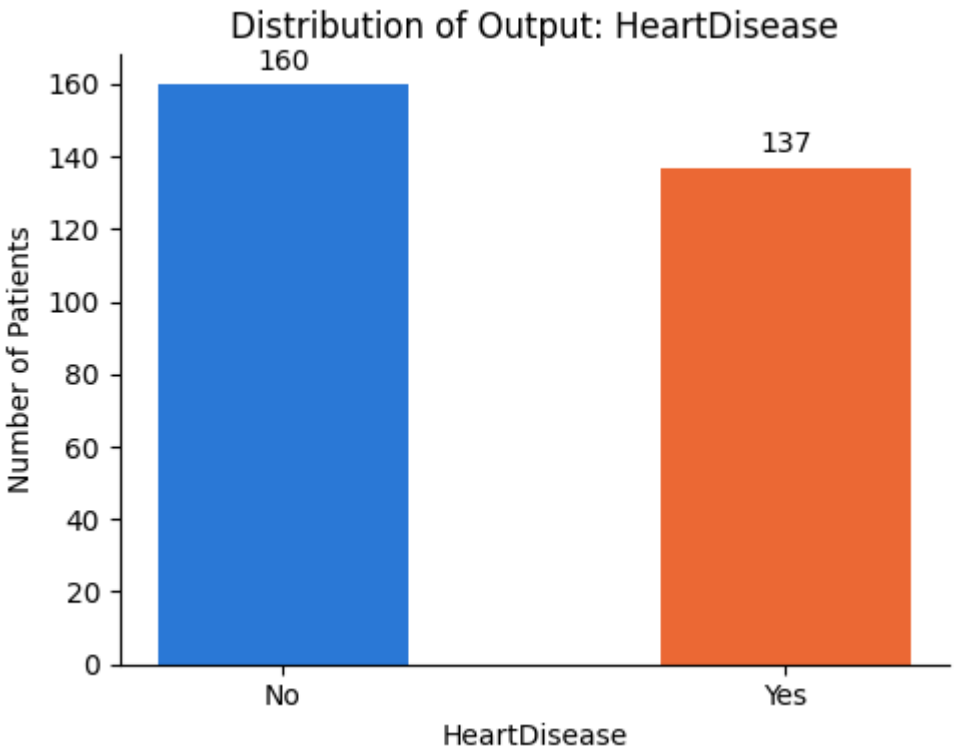
Visualise the data to understand the output distribution and to choose the 4 input attributes.

In [6]:
```
"""
Purpose of this section:
------------------------
Chart 1: How many patients have / do not have heart disease?
This tells us whether the dataset is balanced.
"""

counts = clean_data["HeartDisease"].value_counts().reindex(["No", "Yes"])

fig, ax = plt.subplots(figsize=(5, 4))
bars = ax.bar(counts.index, counts.values, color=[COLOR_NO, COLOR_YES], width=0.5)
ax.bar_label(bars, padding=3)
ax.set_title("Distribution of Output: HeartDisease")
ax.set_xlabel("HeartDisease")
ax.set_ylabel("Number of Patients")
ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.show()
```

```
print(f"No: {counts['No']} ({counts['No'] / counts.sum():.0%}),  Yes: {counts['Yes']} ({counts['Yes'] / counts.sum():.0%})")
print("The dataset is reasonably balanced, so accuracy is a meaningful metric.")
```




```
No: 160 (54%),  Yes: 137 (46%)
The dataset is reasonably balanced, so accuracy is a meaningful metric.
```

In [7]:

```
"""
Purpose of this section:
------------------------
Chart 2: Correlation of every numeric attribute with the output (num > 0).
Attributes with a larger (positive or negative) correlation are more useful
for predicting heart disease. This helps to select the 4 input attributes.
"""

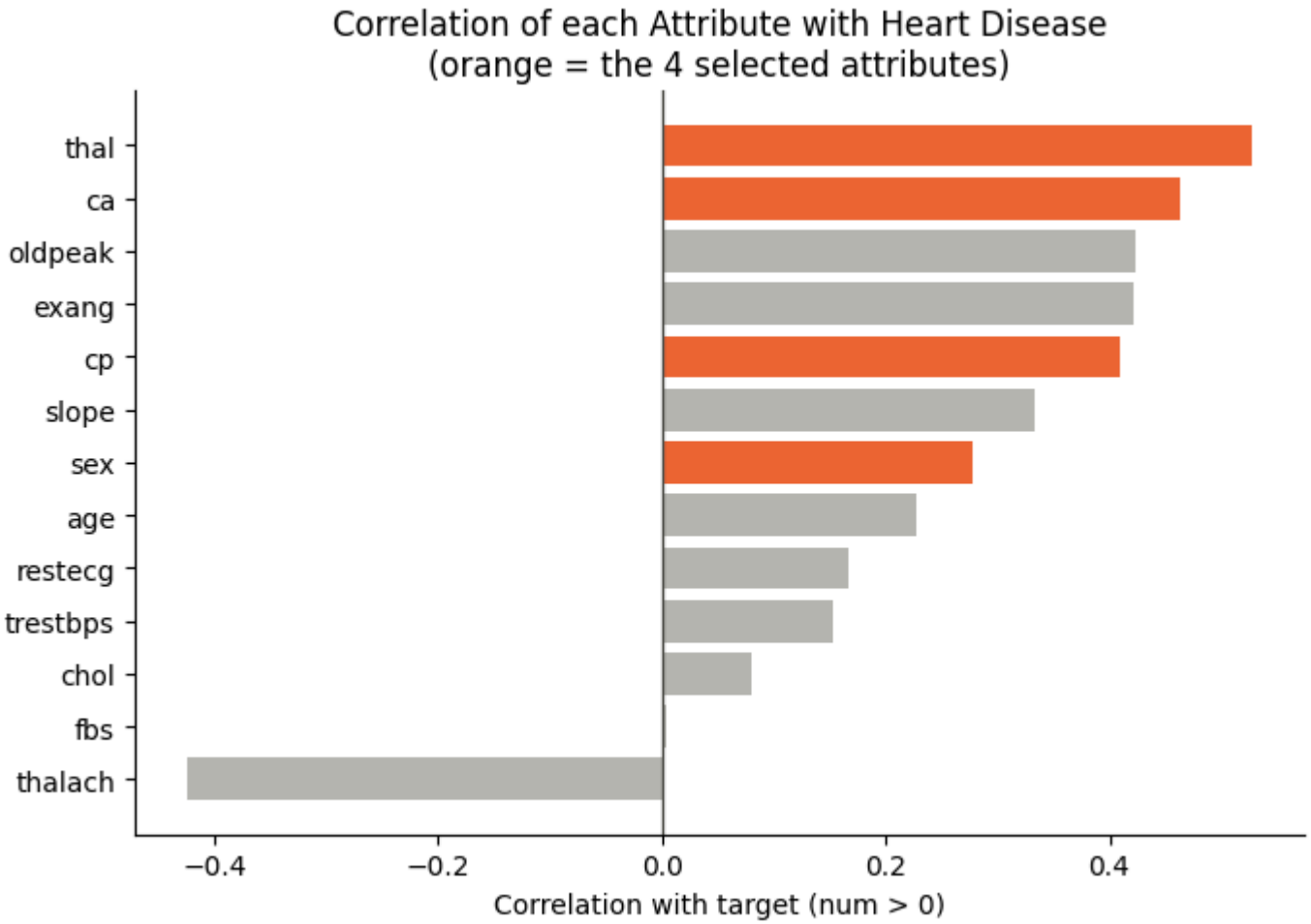
numeric_data = clean_data[["age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
                           "thalach", "exang", "oldpeak", "slope", "ca", "thal"]].copy()
numeric_data["target"] = (clean_data["num"] > 0).astype(int)

target_corr = numeric_data.corr()["target"].drop("target").sort_values()

fig, ax = plt.subplots(figsize=(7, 5))
bar_colors = [COLOR_YES if c in ["sex", "cp", "ca", "thal"] else "#b8b7b1" for c in target_corr.index]
```

```
ax.barh(target_corr.index, target_corr.values, color=bar_colors)
ax.axvline(0, color="#52514e", linewidth=1)
ax.set_title("Correlation of each Attribute with Heart Disease\n(orange = the 4 selected attributes)")
ax.set_xlabel("Correlation with target (num > 0)")
ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.show()

print(target_corr.sort_values(ascending=False).round(2))
```



```
thal        0.53
ca          0.46
oldpeak     0.42
exang       0.42
cp          0.41
slope       0.33
sex         0.28
age         0.23
restecg     0.17
trestbps    0.15
chol        0.08
fbs         0.00
thalach    -0.42
Name: target, dtype: float64
```

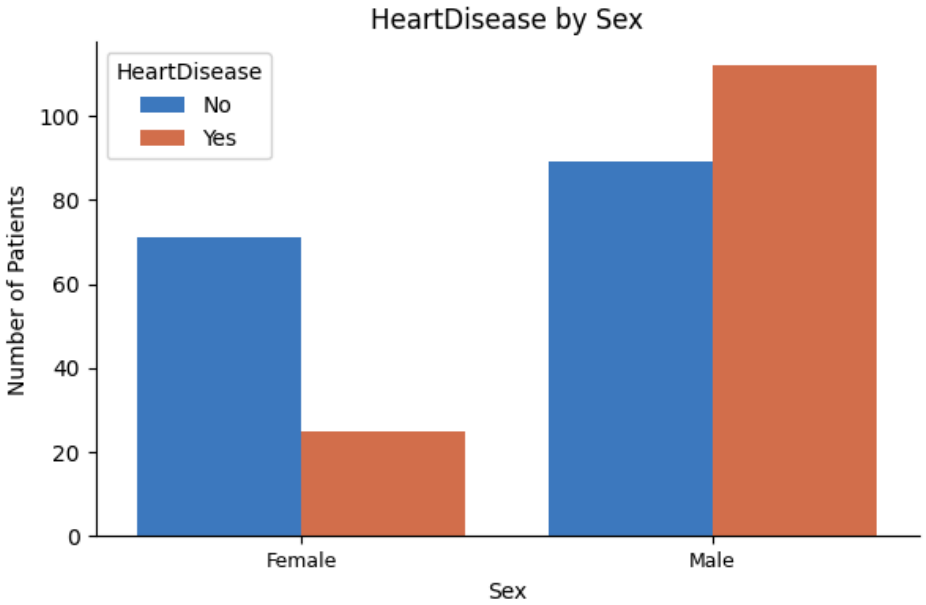
In [8]:

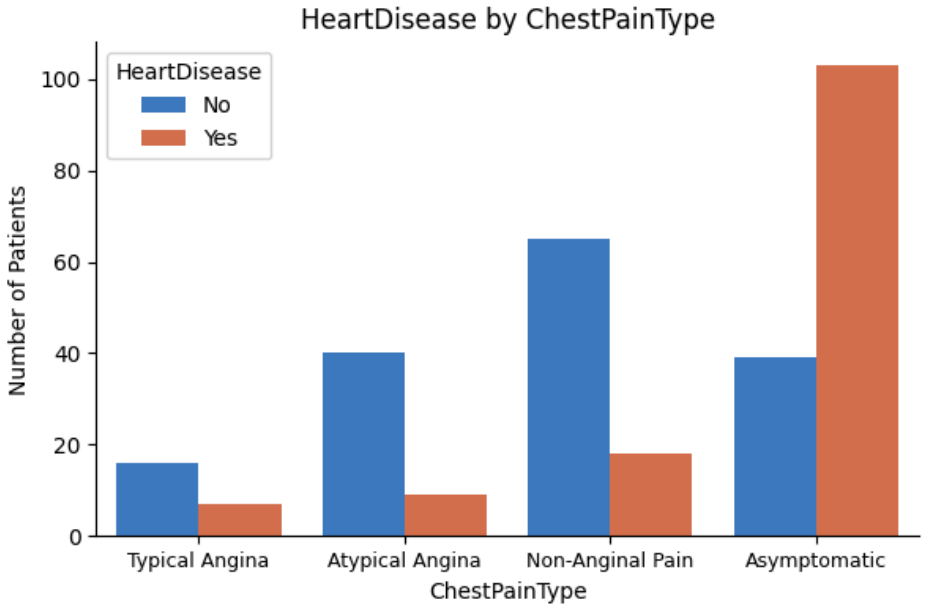
```
"""
Purpose of this section:
------------------------
Chart 3: Number of patients with / without heart disease for each value of
the 4 selected input attributes.
"""

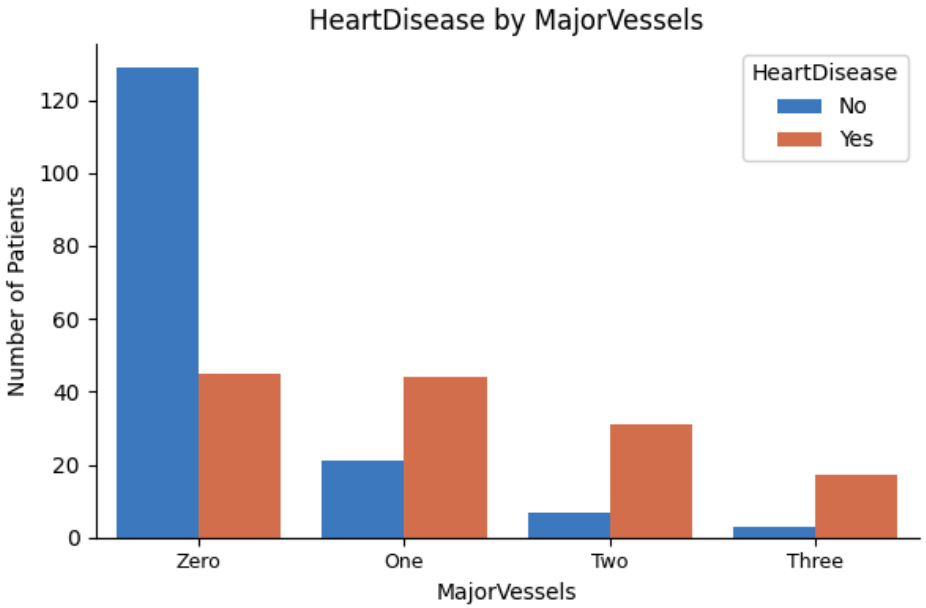
feature_orders = {
    "Sex": ["Female", "Male"],
    "ChestPainType": ["Typical Angina", "Atypical Angina", "Non-Anginal Pain", "Asymptomatic"],
    "MajorVessels": ["Zero", "One", "Two", "Three"],
    "Thal": ["Normal", "Fixed Defect", "Reversible Defect"],
}

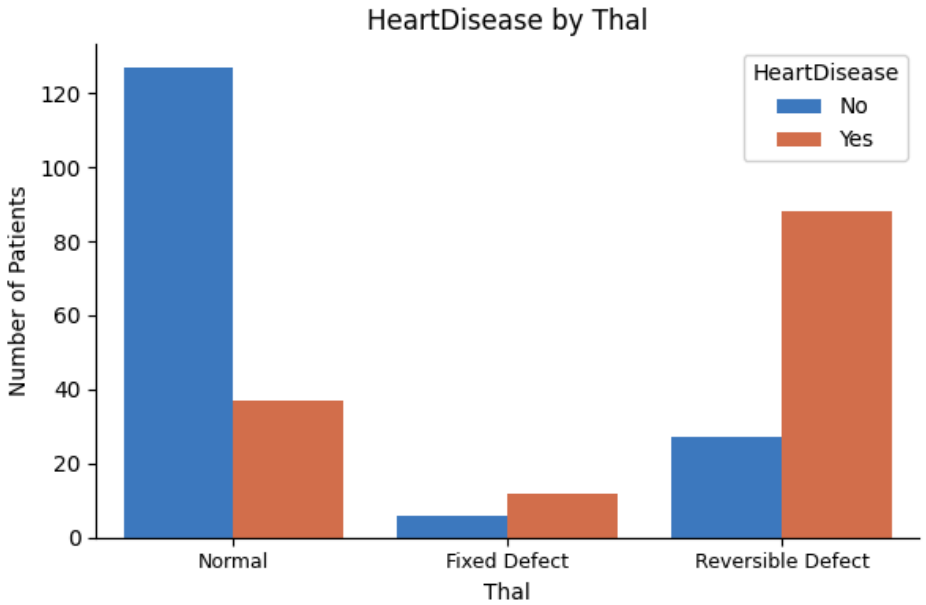
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for ax, (feature, order) in zip(axes.ravel(), feature_orders.items()):
    sns.countplot(data=clean_data, x=feature, hue="HeartDisease", order=order,
                  hue_order=["No", "Yes"], palette=[COLOR_NO, COLOR_YES], ax=ax)
    ax.set_title(f"HeartDisease by {feature}")
    ax.set_ylabel("Number of Patients")
    ax.tick_params(axis="x", labelsize=9)
    ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
plt.show()

# Percentage of patients with heart disease for each value
for feature, order in feature_orders.items():
    rate = (clean_data.groupby(feature)["HeartDisease"]
            .apply(lambda s: (s == "Yes").mean() * 100).reindex(order).round(1))
    print(f"\n% with Heart Disease by {feature}:")
    print(rate.to_string())
```


HeartDisease by Sex
HeartDisease
No
Yes
Number of Patients
0
20
40
60
80
100
Female
Male
Sex


HeartDisease by ChestPainType
HeartDisease
No
Yes
Number of Patients
0
20
40
60
80
100
Typical Angina
Atypical Angina
Non-Anginal Pain
Asymptomatic
ChestPainType


HeartDisease by MajorVessels
HeartDisease
No
Yes
Number of Patients
0
20
40
60
80
100
120
Zero
One
Two
Three
MajorVessels


HeartDisease by Thal
HeartDisease
No
Yes
Number of Patients
0
20
40
60
80
100
120
Normal
Fixed Defect
Reversible Defect
Thal

```
% with Heart Disease by Sex:
Sex
Female    26.0
Male      55.7

% with Heart Disease by ChestPainType:
ChestPainType
Typical Angina      30.4
Atypical Angina     18.4
Non-Anginal Pain    21.7
Asymptomatic        72.5

% with Heart Disease by MajorVessels:
MajorVessels
Zero     25.9
One      67.7
Two      81.6
Three    85.0

% with Heart Disease by Thal:
Thal
Normal               22.6
Fixed Defect         66.7
Reversible Defect    76.5
```

EDA Findings:

- The output is reasonably balanced (≈54% No, ≈46% Yes).
- **Thal**, **MajorVessels (ca)** and **ChestPainType (cp)** are among the attributes most strongly correlated with heart disease, and **Sex** adds useful information (men have a much higher rate of heart disease than women in this dataset).
- Patients with **Asymptomatic** chest pain, a **Reversible Defect** in the thallium test, or **one or more** blocked major vessels have a much higher rate of heart disease.
- `oldpeak` and `exang` are also correlated, but several 4-attribute combinations were compared with 10-fold cross validation and **Sex + ChestPainType + MajorVessels + Thal** was among the best (≈0.80–0.83), while also being easy for a user to enter.
- All 4 attributes are **categorical**, just like the 4 Titanic attributes, so the same Label Encoding approach can be used without any change.

Step 4: Feature Extraction

o Features are already extracted in the UCI dataset

o **Feature Selection:** As required by the assignment, select only **4 input attributes** (Sex, ChestPainType, MajorVessels, Thal) and the output attribute (HeartDisease), based on the EDA above

In [9]:
```
"""
Purpose of this section:
------------------------
This part of the code keeps only the 4 selected input attributes and the
output attribute, in the same format as the Titanic sample data
(4 categorical inputs followed by 1 categorical output).
The result is saved as 'sample-data.csv'.
"""

sample_data = clean_data[["Sex", "ChestPainType", "MajorVessels", "Thal", "HeartDisease"]].copy()

# Save the selected data to a CSV file
sample_data.to_csv(r"sample-data.csv", index=False, header=True)

print("\n\nSample Data with 4 Selected Input Attributes (first 10 rows):")
print("============================================================\n")
print(sample_data.head(10))
print("\nNumber of Instances in Sample Data:", sample_data["Sex"].count())
print("\nSaved: sample-data.csv")
```

```
Sample Data with 4 Selected Input Attributes (first 10 rows):
============================================================

      Sex     ChestPainType MajorVessels              Thal HeartDisease
0    Male    Typical Angina         Zero       Fixed Defect           No
1    Male      Asymptomatic        Three             Normal          Yes
2    Male      Asymptomatic          Two  Reversible Defect          Yes
3    Male  Non-Anginal Pain         Zero             Normal           No
4  Female   Atypical Angina         Zero             Normal           No
5    Male   Atypical Angina         Zero             Normal           No
6  Female      Asymptomatic          Two             Normal          Yes
7  Female      Asymptomatic         Zero             Normal           No
8    Male      Asymptomatic          One  Reversible Defect          Yes
9    Male      Asymptomatic         Zero  Reversible Defect          Yes

Number of Instances in Sample Data: 297

Saved: sample-data.csv
```

Step 5: Label Encoding the Sample Data (Input and Output is converted in Numeric Representation)

Step 5.1: Train the Label Encoder

In [10]:
```
"""
Purpose of this section:
```

```
-------------------------
This part of the code sets up and trains Label Encoders.
Label Encoding is used to convert categorical values (like 'Male', 'Female',
'Asymptomatic', 'Normal') into numeric codes that machine learning algorithms can work with.

Steps:
1. Define label categories for each attribute.
2. Initialize a LabelEncoder for each attribute.
3. Train (fit) each LabelEncoder with its respective category values.
"""

# Define label categories for each feature
# These are possible values each column can have in the dataset
sex = pd.DataFrame({"Sex": ["Male", "Female"]})
chest_pain = pd.DataFrame({"ChestPainType": ["Typical Angina", "Atypical Angina", "Non-Anginal Pain", "Asymptomatic"]})
major_vessels = pd.DataFrame({"MajorVessels": ["Zero", "One", "Two", "Three"]})
thal = pd.DataFrame({"Thal": ["Normal", "Fixed Defect", "Reversible Defect"]})
heart_disease = pd.DataFrame({"HeartDisease": ["Yes", "No"]})

# Create LabelEncoder objects for each categorical column
sex_label_encoder = LabelEncoder()
chest_pain_label_encoder = LabelEncoder()
major_vessels_label_encoder = LabelEncoder()
thal_label_encoder = LabelEncoder()
heart_disease_label_encoder = LabelEncoder()

# Train (fit) the encoders on the given categories
# np.ravel() flattens the DataFrame column into a 1D array for the encoder
sex_label_encoder.fit(np.ravel(sex))
chest_pain_label_encoder.fit(np.ravel(chest_pain))
major_vessels_label_encoder.fit(np.ravel(major_vessels))
thal_label_encoder.fit(np.ravel(thal))
heart_disease_label_encoder.fit(np.ravel(heart_disease))

# Show the numeric code assigned to each category
for name, encoder in [("Sex", sex_label_encoder), ("ChestPainType", chest_pain_label_encoder),
                      ("MajorVessels", major_vessels_label_encoder), ("Thal", thal_label_encoder),
                      ("HeartDisease", heart_disease_label_encoder)]:
    print(f"{name:14s}:", {c: int(v) for c, v in zip(encoder.classes_, encoder.transform(encoder.classes_))})
```

```
Sex           : {'Female': 0, 'Male': 1}
ChestPainType : {'Asymptomatic': 0, 'Atypical Angina': 1, 'Non-Anginal Pain': 2, 'Typical Angina': 3}
MajorVessels  : {'One': 0, 'Three': 1, 'Two': 2, 'Zero': 3}
Thal          : {'Fixed Defect': 0, 'Normal': 1, 'Reversible Defect': 2}
HeartDisease  : {'No': 0, 'Yes': 1}
```

Step 5.2: Label Encode the Output

In [11]:

```
"""
Purpose of this section:
-------------------------
```

```
This part of the code applies Label Encoding to the target/output variable
("HeartDisease") in the dataset. The categorical values ("Yes", "No") are
converted into numerical values (1 and 0) that can be used by
machine learning algorithms.
"""

# Create copies of the dataset (one for encoded output, one to keep original)
sample_data_encoded_output = sample_data.copy()
original_sample_data = sample_data.copy()

# Transform "HeartDisease" (categorical -> numeric) using the trained LabelEncoder
print("\n\nHeartDisease Attribute After Label Encoding (first 10 rows):")
print("============================================================\n")
sample_data["encoded_heart_disease"] = heart_disease_label_encoder.transform(sample_data["HeartDisease"])

# Show original "HeartDisease" values and their numeric encodings
print(sample_data[["HeartDisease", "encoded_heart_disease"]].head(10))

# Replace original categorical output column with encoded values
sample_data_encoded_output[["Sex", "ChestPainType", "MajorVessels", "Thal", "HeartDisease"]] = \
    sample_data[["Sex", "ChestPainType", "MajorVessels", "Thal", "encoded_heart_disease"]]

# Save the encoded dataset to a CSV file
sample_data_encoded_output.to_csv(r"sample-data-encoded-output.csv", index=False, header=True)
print("\nSaved: sample-data-encoded-output.csv")
```

```
HeartDisease Attribute After Label Encoding (first 10 rows):
============================================================

  HeartDisease  encoded_heart_disease
0           No                      0
1          Yes                      1
2          Yes                      1
3           No                      0
4           No                      0
5           No                      0
6          Yes                      1
7           No                      0
8          Yes                      1
9          Yes                      1

Saved: sample-data-encoded-output.csv
```

Step 5.3: Label Encode the Input

In [12]:

```
"""
Purpose of this section:
------------------------
This part of the code applies Label Encoding to the input features
("Sex", "ChestPainType", "MajorVessels", "Thal"). Each categorical value
```

```
is converted into a numeric value so that machine learning models
can process them.
"""

# Create a copy of the dataset (to keep original values safe)
sample_data_encoded = sample_data_encoded_output.copy()

# Encode each input attribute
sample_data_encoded_output["encoded_sex"] = sex_label_encoder.transform(sample_data_encoded_output["Sex"])
sample_data_encoded_output["encoded_chest_pain"] = chest_pain_label_encoder.transform(sample_data_encoded_output["ChestPainType"])
sample_data_encoded_output["encoded_major_vessels"] = major_vessels_label_encoder.transform(sample_data_encoded_output["MajorVessels"])
sample_data_encoded_output["encoded_thal"] = thal_label_encoder.transform(sample_data_encoded_output["Thal"])

# Replace categorical columns with encoded values in the dataset
sample_data_encoded[["Sex", "ChestPainType", "MajorVessels", "Thal", "HeartDisease"]] = \
    sample_data_encoded_output[["encoded_sex", "encoded_chest_pain", "encoded_major_vessels",
                                "encoded_thal", "HeartDisease"]]
sample_data_encoded = sample_data_encoded.astype(int)

# Print encoded dataset (with numeric values instead of categories)
print("\n\nSample Data after Label Encoding (first 10 rows):")
print("================================================\n")
print(sample_data_encoded.head(10))

# Save the encoded dataset to a new CSV file
sample_data_encoded.to_csv(r"sample-data-encoded.csv", index=False, header=True)
print("\nSaved: sample-data-encoded.csv")
```

```
Sample Data after Label Encoding (first 10 rows):
================================================

   Sex  ChestPainType  MajorVessels  Thal  HeartDisease
0    1              3             3     0             0
1    1              0             1     1             1
2    1              0             2     2             1
3    1              2             3     1             0
4    0              1             3     1             0
5    1              1             3     1             0
6    0              0             2     1             1
7    0              0             3     1             0
8    1              0             0     2             1
9    1              0             3     2             1

Saved: sample-data-encoded.csv
```

Step 6: Execute the Training Phase

Step 6.1: Splitting Sample Data into Training Data and Testing Data

Note: The Titanic notebook used `shuffle=False`. Here `shuffle=True` with `stratify` is used, so that the training and testing data both contain the same proportion of Yes / No patients (the UCI file is not guaranteed to be randomly ordered). `random_state=0` keeps the split reproducible.

In [13]:
```
"""
Purpose of this section:
------------------------
This part of the code splits the dataset into Training Data and Testing Data.
- Training Data is used to teach the machine learning model.
- Testing Data is used to check how well the model performs on unseen data.
"""

# Split dataset into training (80%) and testing (20%)
# random_state=0 ensures reproducibility (same split every time)
# stratify keeps the same Yes/No ratio in both training and testing data
training_data_encoded, testing_data_encoded = train_test_split(
    sample_data_encoded,
    test_size=0.2,
    random_state=0,
    shuffle=True,
    stratify=sample_data_encoded["HeartDisease"]
)

# Save the training and testing datasets to CSV files
training_data_encoded.to_csv(r"training-data-encoded.csv", index=False, header=True)
testing_data_encoded.to_csv(r"testing-data-encoded.csv", index=False, header=True)

print("Training Data shape:", training_data_encoded.shape)
print("Testing Data shape:", testing_data_encoded.shape)
```

```
Training Data shape: (237, 5)
Testing Data shape: (60, 5)
```

Step 6.2: Splitting Input Vectors and Outputs / Labels of Training Data

In [14]:
```
"""
Purpose of this section:
------------------------
This part of the code separates the features (input vectors) and
the labels (output values) from the training dataset.
"""

# Extract input vectors (features) from training data
input_vector_train = training_data_encoded.iloc[:, :-1]

# Extract output labels (target variable) from training data
output_label_train = training_data_encoded.iloc[:, -1]
```

```
print("Input vectors (train) shape:", input_vector_train.shape)
print("Output labels (train) shape:", output_label_train.shape)
```

```
Input vectors (train) shape: (237, 4)
Output labels (train) shape: (237,)
```

Step 6.3: Train the Support Vector Classifier

In [15]:

```
"""
Purpose of this section:
------------------------
This part of the code trains a Support Vector Classifier (SVC) on the training data.
The SVC is a type of machine learning model that finds the best boundary
to separate different classes (e.g., heart disease vs. no heart disease).
"""

print("\n\nTraining the Support Vector Classifier on Training Data")
print("=========================================================\n")

# Initialize the Support Vector Classifier with chosen parameters
svc_model = svm.SVC(gamma="auto", random_state=0)

# Train the model using the training data (features and labels)
svc_model.fit(input_vector_train, np.ravel(output_label_train))

# Print trained model details (parameters and values)
print(svc_model)
```

```
Training the Support Vector Classifier on Training Data
=========================================================

SVC(gamma='auto', random_state=0)
```

Step 6.4: Save the Trained Model

In [16]:

```
"""
Purpose of this section:
------------------------
This part of the code saves the trained Support Vector Classifier (SVC)
model to your computer so it can be reused later without retraining.
"""

# Save the trained SVC model to a file named "heart_disease_model.pkl"
with open("heart_disease_model.pkl", "wb") as model_file:
    pickle.dump(svc_model, model_file)
print("Saved: heart_disease_model.pkl")
```

```
Saved: heart_disease_model.pkl
```

Step 7: Execute the Testing Phase

Step 7.1: Splitting Input Vectors and Outputs/Labels of Testing Data

In [17]:
```
"""
Purpose of this section:
------------------------
This part of the code separates the features (inputs) and the labels (outputs)
from the testing dataset.
"""

input_vector_test = testing_data_encoded.iloc[:, :-1]
output_label_test = testing_data_encoded.iloc[:, -1]

print("Input vectors (test) shape:", input_vector_test.shape)
print("Output labels (test) shape:", output_label_test.shape)
```

```
Input vectors (test) shape: (60, 4)
Output labels (test) shape: (60,)
```

Step 7.2: Load the Saved Model

In [18]:
```
"""
Purpose of this section:
------------------------
This part of the code loads the previously saved Support Vector Classifier (SVC)
model from disk into memory.
"""

with open("heart_disease_model.pkl", "rb") as model_file:
    model = pickle.load(model_file)
print("Model loaded from heart_disease_model.pkl")
```

```
Model loaded from heart_disease_model.pkl
```

Step 7.3: Evaluate the Machine Learning Model

Step 7.3.1: Make Predictions with the Trained Model on Testing Data

In [19]:
```
"""
Purpose of this section:
------------------------
```

```
This part of the code evaluates the trained Support Vector Classifier (SVC)
on the testing data.
"""

model_predictions = model.predict(input_vector_test)

pd.options.mode.chained_assignment = None
testing_data_encoded["Predictions"] = model_predictions
testing_data_encoded.to_csv(r"model-predictions.csv", index=False, header=True)

model_predictions = testing_data_encoded
print("\n\nPredictions Returned by heart_disease_model (first 15 test instances):")
print("======================================================================\n")
print(model_predictions.head(15))
print("\nSaved: model-predictions.csv")
```

```
Predictions Returned by heart_disease_model (first 15 test instances):
======================================================================

     Sex  ChestPainType  MajorVessels  Thal  HeartDisease  Predictions
69     1              2             3     1             1            0
2      1              0             2     2             1            1
94     1              0             0     2             1            1
119    1              0             2     2             1            1
266    1              0             0     2             1            1
264    1              0             3     2             1            1
281    1              0             1     0             1            1
36     1              0             3     2             1            1
282    0              0             2     0             1            1
192    1              0             2     1             1            1
271    1              3             3     2             0            0
230    0              1             0     1             0            0
149    1              3             3     2             0            0
132    1              0             3     1             0            0
248    1              0             0     2             1            1

Saved: model-predictions.csv
```

Step 7.4: Calculate the Accuracy Score

In [20]:

```
"""
Purpose of this section:
------------------------
This part of the code calculates how accurate the trained model is on the
testing dataset.
"""

model_accuracy_score = accuracy_score(
    model_predictions["HeartDisease"],
```

```
    model_predictions["Predictions"]
)

print("\n\nAccuracy Score:")
print("===============\n")
print(round(model_accuracy_score, 2))
```

```
Accuracy Score:
===============

0.88
```

Step 7.5: Detailed Evaluation Metrics (Precision, Recall, F1, Confusion Matrix)

In [21]:

```
"""
Purpose of this section:
------------------------
Accuracy alone can be misleading. This section adds the standard classification metrics:
- Precision: of everyone the model predicted as "Heart Disease", how many actually have it?
- Recall: of everyone who actually has heart disease, how many did the model catch?
  (Recall is very important in medicine - a missed patient is a costly error.)
- F1-score: the harmonic mean of precision and recall.
- Confusion Matrix: a table of correct vs. incorrect predictions per class.
"""

print("\nClassification Report:")
print("======================\n")
print(classification_report(
    model_predictions["HeartDisease"],
    model_predictions["Predictions"],
    target_names=["No Heart Disease (0)", "Heart Disease (1)"]
))

conf_matrix = confusion_matrix(model_predictions["HeartDisease"], model_predictions["Predictions"])
conf_matrix_df = pd.DataFrame(
    conf_matrix,
    index=["Actual: No Heart Disease", "Actual: Heart Disease"],
    columns=["Predicted: No Heart Disease", "Predicted: Heart Disease"]
)
print("\nConfusion Matrix:")
print("=================\n")
print(conf_matrix_df)

plt.figure(figsize=(5, 4))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues",
            xticklabels=["No Disease", "Disease"],
            yticklabels=["No Disease", "Disease"])
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
plt.title("Confusion Matrix - SVC on Heart Disease Testing Data")
plt.tight_layout()
plt.show()
```

```
Classification Report:
======================

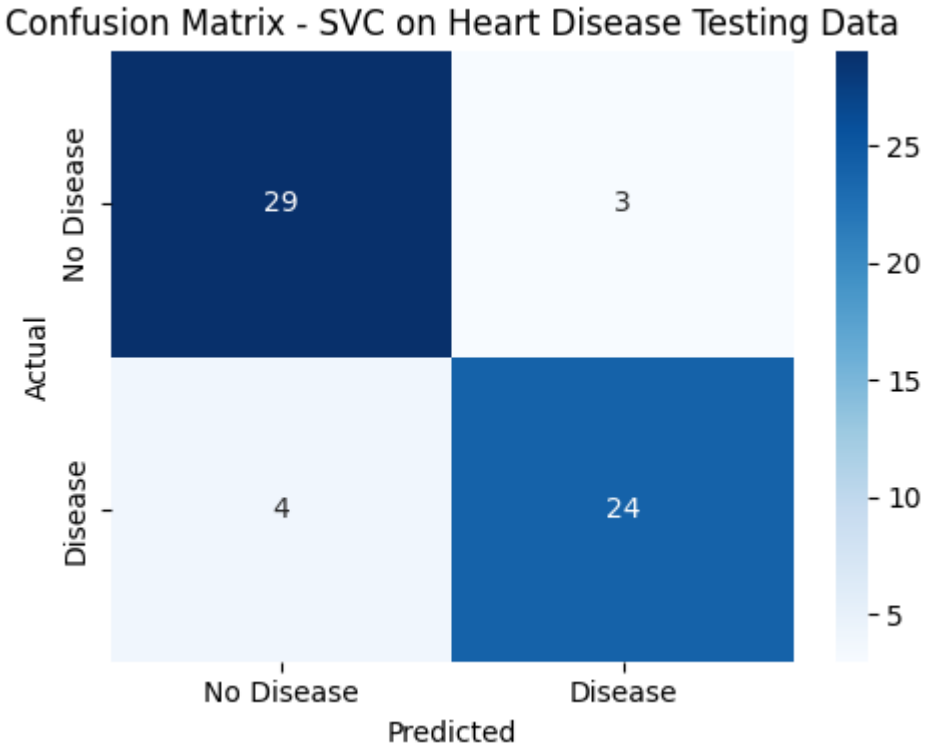
                     precision    recall  f1-score   support

No Heart Disease (0)       0.88      0.91      0.89        32
   Heart Disease (1)       0.89      0.86      0.87        28

            accuracy                           0.88        60
           macro avg       0.88      0.88      0.88        60
        weighted avg       0.88      0.88      0.88        60


Confusion Matrix:
==================

                         Predicted: No Heart Disease  Predicted: Heart Disease
Actual: No Heart Disease                           29                         3
Actual: Heart Disease                               4                        24
```



Step 7.6: 10-Fold Cross Validation (Robustness Check)

In [22]:
```
"""
Purpose of this section:
------------------------
The testing data has only about 60 patients, so a single accuracy score can
change a lot depending on which patients end up in the test set.
10-fold cross validation trains and tests the same SVC 10 times on different
splits of the whole dataset and reports the average accuracy, which is a
more reliable estimate of how the model will perform on new patients.
"""

all_inputs = sample_data_encoded.iloc[:, :-1]
all_outputs = sample_data_encoded.iloc[:, -1]

cv_scores = cross_val_score(svm.SVC(gamma="auto", random_state=0), all_inputs, all_outputs, cv=10)

print("\n\nAccuracy in each of the 10 folds:")
print("=================================\n")
print(np.round(cv_scores, 2))
print(f"\nMean Accuracy (10-fold): {cv_scores.mean():.2f}  (+/- {cv_scores.std():.2f})")
```

```
Accuracy in each of the 10 folds:
=================================

[0.83 0.83 0.83 0.83 0.83 0.73 0.73 0.83 0.76 0.76]

Mean Accuracy (10-fold): 0.80  (+/- 0.04)
```

Step 8: Execute the Application Phase

Step 8.1: Take Input from User

In [23]:
```
"""
Purpose of this section:
------------------------
This part of the code allows the user to enter their own data
(Sex, ChestPainType, MajorVessels, Thal). In an interactive Jupyter session,
running this cell can prompt you for input(). For this notebook copy,
a demo patient is pre-filled so the notebook can also be run
top-to-bottom without needing to type anything.

To try your own values interactively, replace the four lines below with:
    sex_input = input("Please enter Sex here (Male, Female): ").strip()
    chest_pain_input = input("Please enter Chest Pain Type here (Typical Angina, Atypical Angina, Non-Anginal Pain, Asymptomatic): ").strip()
    major_vessels_input = input("Please enter Number of Major Vessels here (Zero, One, Two, Three): ").strip()
```

```
    thal_input = input("Please enter Thal here (Normal, Fixed Defect, Reversible Defect): ").strip()
"""

# Demo patient (edit these values, or swap in input() calls as shown above)
sex_input = "Male"
chest_pain_input = "Asymptomatic"
major_vessels_input = "Two"
thal_input = "Reversible Defect"

print(f"Sex: {sex_input}, ChestPainType: {chest_pain_input}, MajorVessels: {major_vessels_input}, Thal: {thal_input}")
```

```
Sex: Male, ChestPainType: Asymptomatic, MajorVessels: Two, Thal: Reversible Defect
```

Step 8.2: Convert User Input into Feature Vector (Exactly Same as Feature Vectors of Sample Data)

In [24]:

```
"""
Purpose of this section:
-------------------------
This part of the code takes the user's input (entered as text)
and converts it into a structured feature vector (DataFrame).
"""

user_input = pd.DataFrame({
    "Sex": [sex_input],
    "ChestPainType": [chest_pain_input],
    "MajorVessels": [major_vessels_input],
    "Thal": [thal_input]
})

print("\n\nUser Input Feature Vector:")
print("==========================\n")
print(user_input)
```

```
User Input Feature Vector:
==========================

    Sex ChestPainType MajorVessels              Thal
0  Male  Asymptomatic          Two  Reversible Defect
```

Step 8.3: Label Encoding of Feature Vector (Exactly Same as Label Encoded Feature Vectors of Sample Data)

In [25]:

```
"""
Purpose of this section:
-------------------------
This part of the code encodes the user's input into numeric values
```

```
using the previously trained LabelEncoders.
"""

unseen_data_features = user_input.copy()

unseen_data_features["Sex"] = sex_label_encoder.transform(user_input["Sex"])
unseen_data_features["ChestPainType"] = chest_pain_label_encoder.transform(user_input["ChestPainType"])
unseen_data_features["MajorVessels"] = major_vessels_label_encoder.transform(user_input["MajorVessels"])
unseen_data_features["Thal"] = thal_label_encoder.transform(user_input["Thal"])

print("\n\nUser Input Encoded Feature Vector:")
print("==================================\n")
print(unseen_data_features)
```

```
User Input Encoded Feature Vector:
==================================

   Sex  ChestPainType  MajorVessels  Thal
0    1              0             2     2
```

Step 8.4: Load the Saved Model

In [26]:

```
"""
Purpose of this section:
------------------------
This part of the code loads the previously saved Support Vector Classifier (SVC)
model from the .pkl file into memory for the Application Phase.
"""

with open("heart_disease_model.pkl", "rb") as model_file:
    model = pickle.load(model_file)
print("Model loaded from heart_disease_model.pkl")
```

```
Model loaded from heart_disease_model.pkl
```

Step 8.5: Model Prediction

Step 8.5.1: Apply Model on the Label Encoded Feature Vector of unseen instance and return Prediction to the User

In [27]:

```
"""
Purpose of this section:
------------------------
This part of the code uses the trained Support Vector Classifier (SVC)
to predict whether a new patient (user input) has heart disease or not.
"""
```

```
predicted_heart_disease = model.predict(unseen_data_features)[0]

if predicted_heart_disease == 1:
    prediction = "HEART DISEASE: YES"
if predicted_heart_disease == 0:
    prediction = "HEART DISEASE: NO"

pretty_table = PrettyTable()
pretty_table.add_column("       ** Prediction **       ", [prediction])

print(pretty_table)
```

```
+--------------------------------+
|        ** Prediction **        |
+--------------------------------+
|       HEART DISEASE: YES       |
+--------------------------------+
```

In [28]:

```
"""
Purpose of this section:
------------------------
A second example: a low-risk patient, to show that the model returns
both possible outputs.
"""

low_risk_patient = pd.DataFrame({
    "Sex": ["Female"],
    "ChestPainType": ["Non-Anginal Pain"],
    "MajorVessels": ["Zero"],
    "Thal": ["Normal"]
})

low_risk_encoded = low_risk_patient.copy()
low_risk_encoded["Sex"] = sex_label_encoder.transform(low_risk_patient["Sex"])
low_risk_encoded["ChestPainType"] = chest_pain_label_encoder.transform(low_risk_patient["ChestPainType"])
low_risk_encoded["MajorVessels"] = major_vessels_label_encoder.transform(low_risk_patient["MajorVessels"])
low_risk_encoded["Thal"] = thal_label_encoder.transform(low_risk_patient["Thal"])

low_risk_prediction = heart_disease_label_encoder.inverse_transform(model.predict(low_risk_encoded))[0]

pretty_table = PrettyTable()
pretty_table.field_names = ["Sex", "ChestPainType", "MajorVessels", "Thal", "** Prediction **"]
pretty_table.add_row(list(low_risk_patient.iloc[0]) + [f"HEART DISEASE: {low_risk_prediction.upper()}"])
print(pretty_table)
```

```
+--------+------------------+--------------+--------+-------------------+
|  Sex   |  ChestPainType   | MajorVessels |  Thal  |  ** Prediction ** |
+--------+------------------+--------------+--------+-------------------+
| Female | Non-Anginal Pain |     Zero     | Normal | HEART DISEASE: NO |
+--------+------------------+--------------+--------+-------------------+
```

Step 9: Execute the Feedback Phase

Step 9.1: Collect Feedback from Users and Domain Experts on Performance of the Model Deployed in the Real World

In [29]:
```
"""
Purpose of this section:
------------------------
Step 9.1: Example feedback collected from a domain expert (doctor) and
users reviewing this model's predictions on a few cases.
"""

example_feedback = pd.DataFrame({
    "Reviewer": ["Domain Expert (Cardiologist)", "Domain Expert (Cardiologist)", "Student Tester"],
    "Case": [
        "Male, Asymptomatic chest pain, 2 vessels, Reversible Defect -> predicted YES",
        "Female, Non-Anginal Pain, 0 vessels, Normal thal -> predicted NO",
        "Model missed some patients who actually had heart disease (false negatives)",
    ],
    "Feedback": [
        "Matches clinical knowledge: blocked vessels and a reversible thallium defect are strong signs of heart disease.",
        "Matches clinical knowledge: this is a low-risk profile.",
        "In medicine a missed patient (false negative) is more dangerous than a false alarm, so recall for 'Yes' should be improved.",
    ],
})
print(example_feedback.to_string(index=False))
```

```
                   Reviewer                                                                  Case                                                                
```

```
    "Use One-Hot Encoding instead of Label Encoding, so the model does not assume an order between categories like chest pain types",
    "Tune the SVC (C, gamma, kernel) with cross validation and compare with other models (Logistic Regression, Random Forest)",
    "Adjust the decision threshold / class weights to increase recall for the 'Yes' class (fewer missed patients)",
]
for i, item in enumerate(improvements, 1):
    print(f"{i}. {item}")
```

```
1. Add numeric attributes such as Age, Max Heart Rate (thalach) and ST Depression (oldpeak) - they are also strongly related to heart disease
2. Use more data: combine the Cleveland data with the Hungarian, Switzerland and VA Long Beach subsets of the same UCI dataset
3. Use One-Hot Encoding instead of Label Encoding, so the model does not assume an order between categories like chest pain types
4. Tune the SVC (C, gamma, kernel) with cross validation and compare with other models (Logistic Regression, Random Forest)
5. Adjust the decision threshold / class weights to increase recall for the 'Yes' class (fewer missed patients)
```

Step 9.3: Improve the Model Based on Feedback

In [31]:

```
"""
Purpose of this section:
------------------------
Step 9.3: In a real deployment, the next version of this notebook would
retrain with the improvements listed above and re-run Steps 6-7 to confirm
that the new model performs better before replacing the deployed model.
"""

print("Next version: add Age, thalach and oldpeak, use One-Hot Encoding and tune the SVC with cross validation,")
print("then re-evaluate (focusing on recall for 'Yes') before replacing heart_disease_model.pkl")
```

```
Next version: add Age, thalach and oldpeak, use One-Hot Encoding and tune the SVC with cross validation,
then re-evaluate (focusing on recall for 'Yes') before replacing heart_disease_model.pkl
```

Step 10: Deploy the Model to the Cloud (Streamlit Community Cloud)

The trained model (`heart_disease_model.pkl`) is deployed as a public web app, so anyone can use it from a browser without installing Python or Jupyter.

Live App: https://heartdiseaseprediction-omqtad3saltpkdqo4ayxtn.streamlit.app/

Step 10.1: Web App Code

The web app is in `streamlit-app/app.py` of the GitHub repository. It:

1. Loads the trained model `heart_disease_model.pkl` with `pickle.load(...)` .
2. Re-creates the **same LabelEncoders** used in Step 5.1 (fit on the same fixed category lists, so the codes are identical).
3. Uses Streamlit drop-down boxes (`st.selectbox`) to take the 4 inputs from the user.

4. Calls `model.predict(...)` when the **Predict** button is clicked and shows the result.

`streamlit-app/requirements.txt` lists the packages the app needs (streamlit, scikit-learn, pandas, numpy).

Step 10.2: Upload the Project to GitHub

Repository: https://github.com/Hamna-Mubarak/Heart_Disease_Prediction

Step 10.3: Deploy on Streamlit Community Cloud

1. Go to **https://share.streamlit.io** and **Sign in with GitHub**.
2. Click **Create app → Deploy a public app from GitHub**.
3. Repository: `Hamna-Mubarak/Heart_Disease_Prediction`, Branch: `main`, Main file path: `streamlit-app/app.py`.
4. Open **Advanced settings** and choose **Python 3.12**.
5. Click **Deploy** and copy the public `https://....streamlit.app` link.

Conclusion

A complete end-to-end Heart Disease Prediction System was built by following the same steps as the Titanic Passenger Survival Prediction System: the UCI Cleveland Heart Disease data was loaded, understood and pre-processed (missing values removed, output converted to Yes/No, codes converted to readable categories), explored with EDA, reduced to **4 input attributes (Sex, Chest Pain Type, Number of Major Vessels, Thal)**, label encoded, and used to train a Support Vector Classifier. The trained model was saved as `heart_disease_model.pkl`, evaluated on held-out testing data with an **accuracy of 0.88** (precision 0.89 and recall 0.86 for the Heart Disease class, Step 7.4–7.5) and a **10-fold cross validation accuracy of 0.80** (Step 7.6), used to make predictions on new patients, and deployed to the cloud as a Streamlit web app. Even with only 4 attributes, the model captures the main clinical patterns: asymptomatic chest pain, a reversible thallium defect, more blocked major vessels and being male all increase the chance of heart disease.